

🔗 Question

Pregunta 1:

Separar una aplicación en módulos es una tarea importante de arquitectura, una mala separación puede hacer que un pequeño cambio afecte funcionalidades que no debería. Por el contrario una buena separación hace que diferentes equipos de desarrollo puedan trabajar en una misma base de código con baja necesidad de comunicación

Investigue estos dos patrones de descomposición de aplicaciones, el patrón está orientado a el estilo de arquitectura de microservicios que veremos más adelante en el curso. Sin embargo los conceptos mencionados funcionan para Nest dada su capacidad de encapsular aplicaciones en módulos

- [Descomposición por capacidades de negocio](#)
- [Descomposición por subdominios](#)

¿Si usted tuviera que realizar la descomposición de Cheapest, teniendo en cuenta su contexto y madurez, qué patrón escogería?

✓ Success

Respuesta:

Yo escogería la descomposición por subdominios, ya que Cheapest tiene un dominio suficientemente grande y maduro como para identificar contextos con responsabilidades y reglas diferentes, como ventas, logística, inventario, fiado e identificación. Además, la empresa ya tuvo problemas porque un solo cambio resultó costoso y afectó varias funcionalidades. Separar la aplicación de acuerdo con estos subdominios permitiría establecer límites claros, aumentar la cohesión y reducir el impacto de los cambios. Esta separación puede aplicarse inicialmente mediante módulos de Nest dentro del monolito y servir posteriormente como base para extraer microservicios cuando las necesidades de escalabilidad o autonomía lo justifiquen.

🔗 Question

Pregunta 2:

Con sus conocimientos en bases de datos, describa algún caso en donde pueda usar

como táctica de arquitectura reemplazar una base de datos SQL por una no SQL en el contexto de Cheapest. ¿Qué atributos favorecería? ¿Cuáles desfavorecería?

✓ Success

Respuesta:

Para Cheapest, una base de datos NoSQL podría considerarse para el manejo de información de productos y catálogos. Debido a la expansión de Cheapest a diferentes países, los productos pueden requerir atributos adicionales o distintos según el país, sus impuestos, moneda, restricciones comerciales o características particulares. La base de datos no relacional permitiría manejar esta variedad de manera más flexible, ya que los registros no tendrían que compartir exactamente la misma estructura.

Este cambio favorecería principalmente la modificabilidad, debido a que sería más sencillo agregar nuevos atributos o adaptar la información almacenada a las necesidades de cada país sin tener que modificar constantemente un esquema relacional. También podría favorecer la escalabilidad, especialmente teniendo en cuenta el crecimiento esperado de Cheapest y el aumento en la cantidad de productos y tiendas.

Sin embargo, esta decisión podría desfavorecer la consistencia e integridad de los datos, debido a que la flexibilidad del esquema requiere un mayor control desde la aplicación para garantizar que los registros contengan la información necesaria. Además, las consultas que involucren múltiples relaciones entre entidades podrían resultar más complejas que en una base de datos relacional.

? Question

Pregunta 3:

Suponga que en el módulo de Logística y Pedidos se crea un PedidoService que depende de:

- Un RepositorioPedido
- Un ServicioDisponibilidadZona
- Un ServicioCalculoPromociones

Además, la aplicación opera en múltiples países (COP, MXN, BRL) y debe soportar alta concurrencia en quincenas.

Imagine que `ServicioCalculoPromociones` mantiene información temporal del request (por ejemplo, reglas dinámicas por país y tipo de tienda).

- ¿Qué scope (singleton, request, transient) recomendaría para cada uno de estos providers? Justifique su respuesta.
- ¿Qué impacto real tendría esto en memoria y rendimiento bajo alta carga? Suponga que en alta carga se reciben 1000 pedidos por segundo, cual sería la complejidad espacial (Notación Big O) de cada uno de estos scopes en este escenario?

✓ Success

Respuesta:

`RepositorioPedido` debería tener scope *singleton*, pues puede compartir una única instancia y el pool de conexiones entre todos los pedidos, siempre que no almacene estado mutable de una petición. Su complejidad espacial respecto al número de solicitudes es $O(1)$, por lo que el costo de instanciación y memoria es bajo.

`ServicioDisponibilidadZona` también debería ser *singleton* si solamente consulta disponibilidad y no guarda datos particulares del pedido en atributos de la instancia. Así se reutiliza para todas las solicitudes y su complejidad espacial es $O(1)$.

`ServicioCalculoPromociones` debería tener scope *request*, porque mantiene información temporal propia de cada petición, como las reglas aplicables al país y al tipo de tienda. Nest crearía una instancia por cada petición activa y, además, `PedidoService` pasaría a ser *request-scoped* de forma implícita al depender de este servicio. Si hay n solicitudes concurrentes, la complejidad espacial es $O(n)$. Los 1000 pedidos por segundo son una tasa, no equivalen necesariamente a 1000 instancias simultáneas: la concurrencia aproximada depende del tiempo de respuesta. Por ejemplo, a 1000 pedidos por segundo y 0,2 segundos por pedido habría cerca de 200 solicitudes activas. Si aumenta la latencia o se forma una cola, también crecerán el consumo de memoria, la creación de objetos y el trabajo del recolector de basura, lo que puede reducir el rendimiento.

No recomendaría el scope *transient* para estos providers. Este crea una instancia nueva por cada consumidor que lo inyecta: su costo crece con el número de consumidores e instancias activas y, en el peor caso, es $O(n)$ bajo carga. Solo sería apropiado si cada consumidor necesitara estado privado independiente, lo cual no se plantea en este escenario.

🔍 Question

Pregunta 4:

- ¿Qué diferencia existe entre Guards, Interceptors, Pipes y Middleware?
- ¿En qué orden exacto se ejecutan dentro del ciclo de vida de una petición?

✓ Success

Respuesta:

El middleware ejecuta lógica transversal sobre los objetos de solicitud y respuesta antes de que Nest seleccione y ejecute el manejador de la ruta. Se utiliza, por ejemplo, para registros de acceso, CORS o modificación de encabezados.

Los guards deciden si una solicitud puede continuar hacia una ruta. Tienen acceso al contexto de ejecución y suelen encargarse de autenticación, autorización y roles.

Los interceptors envuelven la ejecución del manejador. Pueden ejecutar lógica antes y después de este, medir tiempos, transformar la respuesta, manejar caché o modificar el flujo mediante observables.

Los pipes procesan los argumentos que recibirá el método del controlador. Se usan para transformar datos y validarlos; si la validación falla, lanzan una excepción antes de invocar el método.

El orden general exacto del ciclo de una petición es: middleware → guards → interceptors (antes) → pipes → método del controlador → servicios o providers llamados por el controlador → interceptors (después) → respuesta. Si se produce una excepción no manejada, intervienen los exception filters. Los interceptors ejecutan su fase de retorno en orden inverso, porque envuelven la llamada.

🔍 Question

Pregunta 5:

Note que el módulo exporta `CatalogoService` ¿Cómo funciona el mecanismo de exports e imports entre módulos y qué implicaciones tiene en el acoplamiento? ¿Por qué no se exporta el `CatalogoRepository`?

✓ Success

Respuesta:

En Nest, los providers registrados en un módulo son privados para ese módulo por defecto. El arreglo `exports` define cuáles forman parte de su API pública. Para utilizar `CatalogoService` desde otro módulo, este debe incluirse en `exports` de `LogisticaModule`, y el módulo consumidor debe agregar `LogisticaModule` a su arreglo `imports`. De este modo, el contenedor de inyección de dependencias puede inyectar la misma instancia de `CatalogoService` en los providers del módulo consumidor.

Exportar solo los elementos necesarios reduce el acoplamiento entre módulos. El consumidor conoce el servicio y las operaciones de negocio que este ofrece, pero no los detalles internos con los que se implementan. Si se exportaran muchos providers, otros módulos podrían depender de esas implementaciones y sería más difícil modificarlas sin generar efectos colaterales. También debe evitarse una dependencia circular en la que dos módulos se importen mutuamente; normalmente esto indica que los límites o las responsabilidades deben revisarse.

`CatalogoRepository` no se exporta porque es un detalle de infraestructura y persistencia de `LogisticaModule`. Debe ser utilizado por `CatalogoService` y por otros providers internos del mismo módulo, no directamente por módulos externos. Así, las reglas de negocio y validaciones del servicio no pueden ser omitidas, y la implementación del repositorio o de la base de datos puede cambiar sin afectar a los consumidores. Si otro módulo necesita información del catálogo, debe solicitarla mediante `CatalogoService`.